Project Four Design Document

Group 13

1 Introduction

This design document evaluates the performance of a parallel program implemented using pthreads. The program computes the maximum ASCII value per line from a large dataset. The goal of this project is to analyze how performance changes when we increase thread counts, memory allocation, and number of nodes.

2 Software Architecture

The program uses a shared-memory parallel model. The input file is read into memory and the total number of lines is divided among the threads. Each thread processes a unique collection of lines and computes the maximum ASCII value for each line.

Each thread writes to a unique part of the results array. This means no synchronization mechanisms are needed. This also avoids locking overhead and improves performance. The workload is also uniquely segmented which helps to both minimize scheduling overhead and ensure predictable behavior.

3 Experimental Setup

The tests were conducted on Beocat nodes. Each configuration was run multiple times in order to account for variability during tests.

Parameters tested:

- Threads: 1, 2, 4, 8, 16
- Memory: 64MB, 128MB, 512MB, 1GB, 3GB
- Nodes: 1, 2, 4, 8

Metrics collected include execution time and CPU utilization.

4 Results

4.1 Thread Scaling

Threads	Avg Time (s)	Notes
1	24.0	High variance
2	42.0	Worse than 1 thread
4	2.3	Large improvement
8	2.05	Slight improvement
16	1.9	Minimal gain

4.2 Memory Scaling

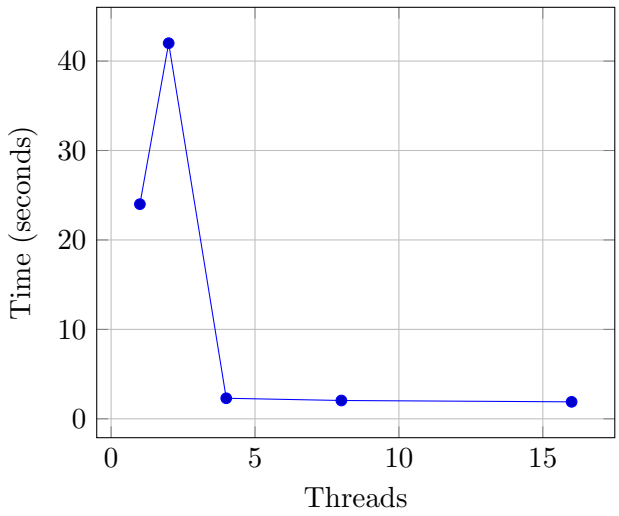
Memory	Avg Time (s)
64MB	6–10
128MB	10–28 (unstable)
512MB	1.95
1GB	1.95
3GB	1.93

4.3 Node Scaling

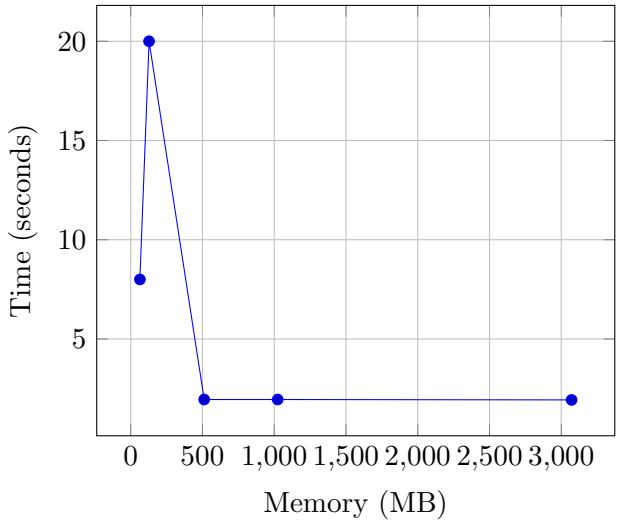
Nodes	Avg Time (s)
1	2.0
2	1.92
4	2.0
8	1.95

5 Graphs

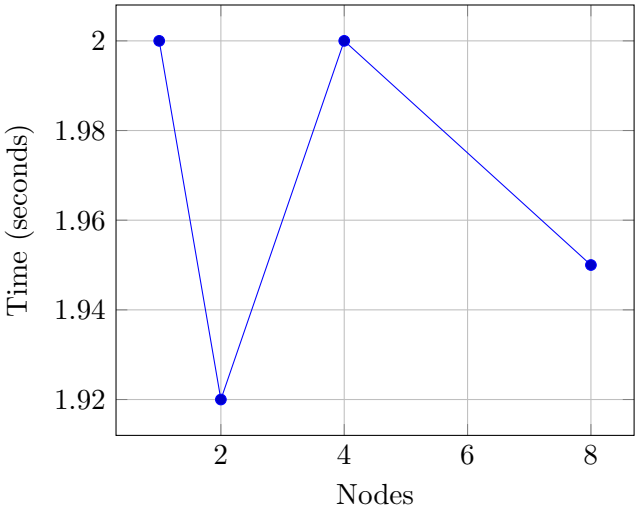
5.1 Threads vs Time



5.2 Memory vs Time



5.3 Nodes vs Time



6 Analysis

6.1 Thread Scaling

The performance of the program noticeably improves from 1 thread to 4 threads; however, there is a strange decrease in performance when going from 1 thread to 2 threads. The 2 thread configuration performed quite poorly. This could be due to scheduling overhead, cache contention, or possibly inefficient thread placement.

Using more than 4 threads does not appear to show that noticeable or significant of performance improvements. This suggests that the system reaches a limit in terms of parallel scalability.

6.2 Memory Scaling

Memory allocation has a major impact on performance. Low memory configurations result in significantly slower and unstable execution times. This is especially evident in the runs using 128MB of memory where the recorded average time is greater than 10 seconds. This suggests memory pressure or paging effects.

Once sufficient memory is available, about 512MB or more in our case, the performance stabilizes and remains consistent.

6.3 Node Scaling

Increasing the number of nodes does not improve performance at all. The average time values jump around randomly from run to run. This is expected because pthreads use a shared-memory model and cannot leverage distributed systems.

6.4 CPU Utilization

CPU utilization increases with thread count. We see the CPU approaching full utilization at higher thread counts. The lower utilization seen in some runs indicates inefficiencies due to memory or scheduling.

7 Conclusion

The pthread implementation demonstrates strong performance improvements with increasing thread counts up to a reasonable level. However, the scalability of the program performance seems limited by memory bandwidth and hardware constraints.

Future work includes comparing performance with the MPI and OpenMP implementations.

8 MPI Implementation and Performance Evaluation

8.1 MPI Design Overview

The MPI implementation uses a distributed memory model. Instead of being divided by threads, the dataset is divided by processes. Each MPI process must compute the max ASCII value for its assigned portion of the input

The program follows this structure:

- The root process reads and distributes input data using MPI communication.
- Each process computes results locally on its subset of lines.
- Results are gathered back to the root process using mpi gather

MPI requires explicit communication between processes. This makes MPI different from pthreads and it introduces communication overhead but enables scaling across multiple nodes.

8.2 Task Scaling (Processes)

Processes	Avg Time (s)	CPU Utilization (%)
1	∼0.23	∼42
2	∼0.30	∼32
4	∼0.30	∼32
8	∼0.35	∼28
16	∼0.40–0.47	∼20

Increasing the number of MPI processes does not improve performance. In fact, runtime slightly increases as more processes are added. This could be caused by:

- Communication overhead between processes
- Increased synchronization costs
- Smaller workloads per process leading to inefficiency

8.3 Memory Scaling

Memory	Avg Time (s)
64MB	∼0.34
128MB	∼0.35 (with outliers up to 2.8s)
512MB	∼0.33
1GB	∼0.33
3GB	∼0.30–0.50 (high variance)

Memory size appears to have minimal impact once sufficient memory is available. On the other hand, variability increases at higher memory allocations due to shared cluster conditions. Some long runtimes correspond to very low CPU utilization. This seems to suggest there is resource contention or scheduling delays.

8.4 Node Scaling

Nodes	Avg Time (s)
1	∼0.31
2	∼0.40–0.42
4	∼0.31–0.36
8	∼0.32–0.36

Increasing the number of nodes does not appear to dramatically improve performance. While MPI enables distributed execution, the workload in this application seems to be too small to benefit from multi node scaling.

8.5 CPU Utilization Analysis

CPU utilization generally decreases as the number of processes increases. This could be due to:

- Increased idle time due to communication

- Load imbalance across processes
- Overhead dominating computation at higher process counts.

Runs with extremely low CPU usage (e.g., 3–4%) correspond to significantly longer runtimes which indicates system noise or contention on the shared cluster

8.6 Comparison: MPI vs Pthreads

Metric	Pthreads	MPI
Model	Shared memory	Distributed memory
Best Runtime	∼1.9s	∼0.23s
Scaling Efficiency	Good up to 4 threads	Poor beyond 1 process
Node Utilization	Not applicable	Supported but not great
Overhead	Low	High (communication)

Key Differences:

- Pthreads performs better for this workload because it avoids communication overhead and operates in shared memory.
- MPI introduces communication and synchronization costs that outweigh benefits for small or moderately sized problems
- MPI shows no meaningful scaling across nodes, confirming that the workload is not large enough to benefit from distributed execution.

8.7 Discussion

The results indicate that MPI is not well-suited for this specific problem at the tested scale. The computation per process is too small to justify the communication overhead.

Performance variability is also higher in MPI runs due to:

- Network latency
- Shared cluster contention (Beocat environment)
- Process scheduling differences across nodes

8.8 Conclusion

The MPI implementation does not outperform the pthreads implementation for this workload. While MPI enables distributed execution, its benefits are only justifiable for significantly larger datasets or more computation heavy tasks

Future improvements could include:

- Increasing problem size to better utilize distributed resources
- Reducing communication frequency